```
# ============================================================
# EXPERIMENT NO. 1
# Real-World Data Collection, Cleaning and Preprocessing
# using Python
# Dataset: Titanic Survival Dataset
# ============================================================

# ------------------------------------------------------------
# 1. IMPORT REQUIRED LIBRARIES
# ------------------------------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

print("Libraries imported successfully!")
print("=" * 60)


# ------------------------------------------------------------
# 2. LOAD DATASET
# ------------------------------------------------------------

df = pd.read_csv("Titanic-Dataset.csv")

print("Dataset loaded successfully!")
print("=" * 60)


# ------------------------------------------------------------
# 3. INSPECT DATASET
# ------------------------------------------------------------

print("\nFIRST 5 ROWS:")
display(df.head())

print("\nDATASET INFORMATION:")
df.info()

print("\nSTATISTICAL SUMMARY:")
display(df.describe())


# ------------------------------------------------------------
# 4. CHECK MISSING VALUES
# ------------------------------------------------------------

print("\nMISSING VALUES BEFORE CLEANING:")
print(df.isnull().sum())


# ------------------------------------------------------------
# 5. HANDLE MISSING VALUES
# Age -> Median
# Embarked -> Mode
# Cabin -> Removed because many values are missing
# ------------------------------------------------------------

df["Age"] = df["Age"].fillna(df["Age"].median())

df["Embarked"] = df["Embarked"].fillna(
    df["Embarked"].mode()[0]
)

df = df.drop(columns=["Cabin"])

print("\nMISSING VALUES AFTER CLEANING:")
print(df.isnull().sum())


# ------------------------------------------------------------
# 6. REMOVE DUPLICATE RECORDS
# ------------------------------------------------------------

print("\nDUPLICATE RECORDS BEFORE REMOVAL:")
print(df.duplicated().sum())

df = df.drop_duplicates()
```

```
print("\nDUPLICATE RECORDS AFTER REMOVAL:")
print(df.duplicated().sum())


# ------------------------------------------------------------
# 7. STANDARDIZE CATEGORICAL VALUES
# ------------------------------------------------------------

df["Sex"] = df["Sex"].str.lower().str.strip()

df["Embarked"] = df["Embarked"].str.upper().str.strip()

print("\nSTANDARDIZED CATEGORICAL VALUES:")
print("Sex:", df["Sex"].unique())
print("Embarked:", df["Embarked"].unique())


# ------------------------------------------------------------
# 8. FEATURE ENGINEERING
# Family Size = SibSp + Parch + 1
# ------------------------------------------------------------

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

print("\nFAMILY SIZE:")
display(df[["SibSp", "Parch", "FamilySize"]].head())


# ------------------------------------------------------------
# 9. CREATE AGE GROUP
# ------------------------------------------------------------

df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0, 12, 18, 60, np.inf],
    labels=["Child", "Teenager", "Adult", "Senior"]
)

print("\nAGE GROUP:")
display(df[["Age", "AgeGroup"]].head())


# ------------------------------------------------------------
# 10. LABEL ENCODING
# Convert categorical values into numerical values
# ------------------------------------------------------------

le = LabelEncoder()

df["Sex"] = le.fit_transform(df["Sex"])

df["Embarked"] = le.fit_transform(df["Embarked"])

df["AgeGroup"] = le.fit_transform(
    df["AgeGroup"].astype(str)
)

print("\nDATA AFTER LABEL ENCODING:")
display(df.head())


# ------------------------------------------------------------
# 11. OUTLIER DETECTION USING BOX PLOT
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

sns.boxplot(x=df["Age"])

plt.title("Box Plot of Age")
plt.xlabel("Age")

plt.show()


plt.figure(figsize=(8, 5))

sns.boxplot(x=df["Fare"])

plt.title("Box Plot of Fare")
plt.xlabel("Fare")

plt.show()
```

```
plt.show()


# ------------------------------------------------------------
# 12. IQR METHOD FOR OUTLIERS
# ------------------------------------------------------------

Q1 = df["Fare"].quantile(0.25)

Q3 = df["Fare"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR

upper_limit = Q3 + 1.5 * IQR

print("\nIQR OUTLIER DETECTION:")
print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)
print("Lower Limit =", lower_limit)
print("Upper Limit =", upper_limit)


# ------------------------------------------------------------
# 13. REMOVE FARE OUTLIERS
# ------------------------------------------------------------

before = len(df)

df = df[
    (df["Fare"] >= lower_limit) &
    (df["Fare"] <= upper_limit)
]

after = len(df)

print("\nOUTLIER TREATMENT:")
print("Rows before:", before)
print("Rows after:", after)
print("Outliers removed:", before - after)


# ------------------------------------------------------------
# 14. STANDARDIZATION
# ------------------------------------------------------------

scaler = StandardScaler()

df[["Age", "Fare"]] = scaler.fit_transform(
    df[["Age", "Fare"]]
)

print("\nDATA AFTER STANDARDIZATION:")
display(df[["Age", "Fare"]].head())


# ------------------------------------------------------------
# 15. FINAL DATASET
# ------------------------------------------------------------

print("\n" + "=" * 60)

print("FINAL CLEANED AND PREPROCESSED DATASET")

print("=" * 60)

display(df.head())

print("\nFINAL DATASET SHAPE:")
print(df.shape)

print("\nFINAL MISSING VALUES:")
print(df.isnull().sum())

print("\nFINAL DUPLICATE RECORDS:")
print(df.duplicated().sum())


# ------------------------------------------------------------
# 16. SAVE CLEANED DATASET
# ------------------------------------------------------------
```

```
output_file = "Titanic_Cleaned_Preprocessed.csv"

df.to_csv(output_file, index=False)

print("\n" + "=" * 60)

print("EXPERIMENT COMPLETED SUCCESSFULLY!")

print("=" * 60)

print("Cleaned dataset saved as:", output_file)


# ============================================================
# QUESTIONS AND ANSWERS
# ============================================================

# Q1. Why is data preprocessing considered one of the most
# important phases in data analytics?
#
# Answer:
# Data preprocessing improves the quality of raw data by
# handling missing values, duplicates, inconsistent data
# and outliers. Clean data gives more accurate analysis
# and better machine learning results.


# Q2. Explain different methods of handling missing values
# with suitable examples.
#
# Answer:
# Missing values can be handled using:
# 1. Mean - replace missing numerical values with mean.
# 2. Median - useful when data contains outliers.
# 3. Mode - used for categorical data.
# 4. Row removal - remove rows containing missing values.
# 5. Column removal - remove columns having many missing values.
#
# Example:
# df["Age"] = df["Age"].fillna(df["Age"].median())


# Q3. Differentiate between Label Encoding and
# One-Hot Encoding.
#
# Answer:
# Label Encoding converts categories into numerical values.
# Example: Male = 1, Female = 0.
#
# One-Hot Encoding creates separate columns for each category.
# It is useful for nominal categorical data.


# Q4. What are outliers? How can they affect analytical results?
#
# Answer:
# Outliers are values that are significantly different from
# other observations. They can affect mean and standard deviation,
# distort graphs and may reduce machine learning performance.


# Q5. Explain the difference between normalization
# and standardization.
#
# Answer:
# Normalization usually scales values between 0 and 1.
#
# Formula:
# X' = (X - Xmin) / (Xmax - Xmin)
#
# Standardization converts data to mean 0 and standard
# deviation 1.
#
# Formula:
# Z = (X - Mean) / Standard Deviation


# Q6. Why should duplicate records be removed before analysis?
#
# Answer:
# Duplicate records can make some observations appear more
# frequently than they actually are. This can produce biased
# statistics and incorrect analytical results.
```

```
# Q7. What is feature engineering? Give two practical examples.
#
# Answer:
# Feature engineering means creating new useful features
# from existing data.
#
# Example 1:
# FamilySize = SibSp + Parch + 1
#
# Example 2:
# Create Age Group such as Child, Teenager, Adult and Senior.


# Q8. Which preprocessing techniques would you apply to the
# IBM HR Employee Attrition dataset and why?
#
# Answer:
# 1. Missing value treatment - to handle incomplete data.
# 2. Duplicate removal - to avoid repeated employee records.
# 3. Categorical encoding - to convert categorical data
#    into numerical form.
# 4. Outlier detection - for Age and MonthlyIncome.
# 5. Standardization - to bring numerical features to
#    a common scale.
# 6. Feature engineering - to create useful features such
#    as Income Category.


# Q9. How does poor-quality data affect machine learning
# model performance?
#
# Answer:
# Poor-quality data can produce incorrect patterns, biased
# predictions, lower accuracy and unreliable results.
# Missing values, duplicates, outliers and inconsistent
# data can negatively affect model training.


# Q10. Name any three Python libraries commonly used
# for data preprocessing.
#
# Answer:
# 1. Pandas
# 2. NumPy
# 3. Scikit-learn
#
# ============================================================
# END OF EXPERIMENT
# ============================================================
```

```
Libraries imported successfully!
============================================================
Dataset loaded successfully!
============================================================

FIRST 5 ROWS:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
DATASET INFORMATION:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

STATISTICAL SUMMARY:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000